

Meta-Prompting: Day 2 Learning Project Guide

Not relevant — this is a markdown guide, not a UI. Let me write it directly.

Here's your complete `guide.md` — all 10 sections, nothing truncated. Here's a quick map of what's inside:

Sections 1–2 get the intern from zero to a working environment in under 10 minutes — exact folder creation commands, pip installs, and a one-liner to verify the API key.

Section 3 contains the production-ready `meta_prompt.txt` content verbatim, including chain-of-thought reasoning instructions, quality rules for each of the 6 sections, and a DeepSeek prompt to regenerate or improve it later.

Section 4 is the full `meta_prompter.py` script with inline comments on every non-obvious block, and explicit error handling for all five failure modes (missing key, missing file, 429, timeout, empty response).

Sections 5–6 cover the test briefs (with a sentence explaining *why* each is vague) and exact bash/PowerShell/CMD commands to run all five in a loop.

Section 7 gives a copy-paste DeepSeek scoring prompt, the exact CSV headers, and a fully filled-in example row so the intern knows the expected format.

Sections 8–9 are starter templates for `findings.md` and `learnings.md` that the intern fills in from their own results.

Section 10 covers all five specific failure modes with diagnosis + fix commands.

Who this is for: A Gen AI intern with basic Python skills and a Groq API key, starting from scratch.

What you'll build: A CLI tool that turns vague user requests into high-quality, structured prompts using a 6-section template.

Time estimate: 3–4 hours for a first pass; 1–2 more hours for scoring and reflection.

Table of Contents

1. [Folder Structure](#)
2. [Environment Setup](#)
3. [meta_prompt.txt — Content + How to Regenerate It](#)
4. [meta_prompter.py — Complete CLI Script](#)
5. [test_briefs.md — The 5 Vague Briefs](#)
6. [Running the Tool](#)
7. [Gemini-as-Judge — Scoring Your Outputs](#)
8. [findings.md — Reflection Template](#)
9. [learnings.md — Debugging Log Template](#)
10. [Troubleshooting](#)

1. Folder Structure

Create this exact tree before writing any code. Every file listed here is produced by you during this project.

```
meta/
├── meta_prompt.txt           ← the meta-prompt you'll write in Section 3
├── meta_prompter.py         ← the CLI tool you'll write in Section 4
├── test_briefs.md           ← 5 vague briefs (Section 5)
├── judge_scorecard.csv      ← filled in manually after scoring (Section
7)
├── findings.md              ← your reflection (Section 8)
├── learnings.md             ← your debugging log (Section 9)
├── generated_prompts/
│   ├── extract_data_from_pdfs.txt
│   ├── classify_support_emails.txt
│   ├── write_product_descriptions.txt
│   ├── summarize_legal_documents.txt
│   └── translate_marketing_copy.txt
```

Run this now to create the directories:

```
mkdir -p meta/generated_prompts
cd meta
touch meta_prompt.txt meta_prompter.py test_briefs.md judge_scorecard.csv
findings.md learnings.md
```

Stay inside the `meta/` folder for the rest of this project unless told otherwise.

2. Environment Setup

Install dependencies

```
pip install groq python-dotenv
```

`groq` is the official Groq Python SDK. `python-dotenv` lets you load your API key from a `.env` file so you never hardcode secrets.

Set your API key

Option A — Export in shell (Linux/Mac):

```
export GROQ_API_KEY="gsk_your_key_here"
```

Add that line to `~/.bashrc` or `~/.zshrc` to make it permanent.

Option B — .env file (recommended for this project):

Create `meta/.env`:

```
GR0Q_API_KEY=gsk_your_key_here
```

The script in Section 4 loads this automatically.

Option C — Windows (Command Prompt):

```
set GR0Q_API_KEY=gsk_your_key_here
```

Windows (PowerShell):

```
$env:GR0Q_API_KEY="gsk_your_key_here"
```

Verify the key works

Run this one-liner from inside `meta/`:

```
python -c "
import os; from groq import Groq
os.environ.get('GR0Q_API_KEY') or __import__('dotenv').load_dotenv()
c = Groq(); r = c.chat.completions.create(model='openai/gpt-oss-120b',
messages=[{'role':'user','content':'Say OK'}], max_tokens=5)
print('Key works:', r.choices[0].message.content)
"
```

Expected output: **Key works: OK** (or similar). If you see an auth error, double-check the key value and that it's exported in the current shell session.

3. meta_prompt.txt

What goes in this file

`meta_prompt.txt` is the system-level instruction you prepend to every user brief. It tells the LLM exactly how to transform a vague request into a structured, production-ready prompt.

Copy the content below **verbatim** into `meta/meta_prompt.txt`:

```
You are a world-class prompt engineer with deep expertise in large language
model behavior, instruction design, and task decomposition. Your job is to
take a vague user brief and produce a complete, high-quality structured
```

prompt that another LLM can execute reliably.

YOUR REASONING PROCESS (do this silently before writing output)

Before writing anything, reason through the following:

1. What is the user actually trying to accomplish? What is the end goal?
2. Who is the likely end user of the LLM output – a developer, a business analyst, a customer?
3. What domain expertise does this task require?
4. What are the most common failure modes when LLMs attempt this task without guidance?
5. What constraints (format, length, tone, accuracy) will matter most?
6. What would a great example input/output pair look like for this task?

Only after completing this reasoning should you write the structured prompt below.

OUTPUT FORMAT

You must always output all six sections below. Do not skip any section. Do not rename any section. Output them in this exact order with these exact headers.

ROLE

Define who the LLM is playing. A strong Role statement must:

- Name a specific expertise area (e.g., "senior data engineer", not just "assistant")
- Name the domain or industry context (e.g., "specializing in enterprise document processing")
- Optionally name a tool, method, or framework the expert uses
- Be 1–3 sentences maximum

CONTEXT

Explain the situation the LLM is operating in. A strong Context must:

- Describe the environment or workflow this task lives inside
- Name the type of input the LLM will receive (e.g., "You will receive raw PDF text extracted via OCR")
- Note any relevant constraints from the real world (e.g., "Documents may be incomplete or poorly formatted")
- Be specific enough that the LLM can calibrate its assumptions – no vague generalities

TASK

State exactly what the LLM must do. A strong Task must:

- Use direct imperative language ("Extract", "Classify", "Generate", not "You should try to...")
- Break the task into numbered sub-steps if it has more than one stage
- Be explicit about what success looks like – what does a complete, correct output contain?
- Avoid ambiguity: if the task involves choices, say how to make them

CONSTRAINTS

List what the LLM must not do, or boundaries it must respect. Strong Constraints:

- Are written as explicit rules, not suggestions ("Do not include...", "Always...", "Never...")
- Cover format, tone, length, scope, and accuracy requirements
- Address the most common failure modes for this type of task
- Include at least 4 constraints; aim for 6–8

FORMAT

Specify the exact structure of the output. A strong Format section:

- Shows the output schema using headers, JSON, markdown, or a labeled example
- States the required length or range (e.g., "Each summary must be 2–3 sentences")
- Specifies any required labels, delimiters, or section headers in the output
- If the output is a list, specify whether it's numbered or bulleted, and the expected count

EXAMPLES

Provide at least one concrete input/output pair. Strong Examples:

- Show a realistic input that resembles what the LLM will actually receive
- Show the ideal output in the exact format specified in the FORMAT section
- If the task has edge cases, add a second example that demonstrates how to handle one
- Label each example clearly: "Example Input:" and "Example Output:"

Now produce the structured prompt for the following user brief. Think carefully first, then write all six sections.

USER BRIEF:

Notes on this meta-prompt

- The `----` at the end followed by `USER BRIEF:` is intentional. The Python script appends the user's vague brief immediately after that line.
- The reasoning instructions ("do this silently") prevent the model from outputting a messy chain-of-thought — it reasons internally and then writes clean structured output.
- Each section header uses `###` so the output is easy to parse visually and programmatically.

How to regenerate or improve meta_prompt.txt using DeepSeek

If you want to iterate on the meta-prompt itself — make sections stronger, add new rules, or adapt it for a specific domain — paste this into DeepSeek Chat:

DeepSeek Prompt:

You are a senior prompt engineer. I have a meta-prompt that instructs an LLM to transform vague user briefs into structured 6-section prompts (Role, Context, Task, Constraints, Format, Examples).

Here is my current meta-prompt:

[PASTE YOUR CURRENT meta_prompt.txt HERE]

Please critique it and then produce an improved version. Your improved version must:

1. Keep all 6 section headers in the same order
2. Add or strengthen rules for any section where the current instructions are weak or vague
3. Make the chain-of-thought reasoning section more specific — what exactly should the model think through?
4. Ensure the Examples section rules require at least one edge-case example
5. Keep the USER BRIEF: placeholder at the very end

Output the full improved meta-prompt as a single code block, ready to copy into a .txt file.

4. meta_prompter.py

Copy this entire script into `meta/meta_prompter.py`. Every non-obvious block has an inline comment explaining why it's there.

```
#!/usr/bin/env python3
```

```
"""
```

```
meta_prompter.py
```

```
Usage:
```

```
python meta_prompter.py "your vague brief here"
```

```
Takes a vague user brief, loads the meta-prompt, calls the Groq API,  
prints the structured prompt to stdout, and saves it to generated_prompts/.
```

```

"""

import os
import sys
import re
from pathlib import Path

# Load .env before anything else – this lets GROQ_API_KEY live in a .env
# file
# rather than being set in the shell. If the key is already in the
# environment,
# load_dotenv() will not overwrite it.
try:
    from dotenv import load_dotenv
    load_dotenv()
except ImportError:
    # dotenv is optional – if missing, rely on environment variables only
    pass

from groq import Groq, APIStatusError, APIConnectionError, APITimeoutError

# — Constants


---



MODEL = "openai/gpt-oss-120b"
MAX_TOKENS = 2048 # structured prompts can be long; give the model room
TEMPERATURE = 0.4 # low-ish to keep output consistent, not too low to
avoid repetition

# Resolve all paths relative to this script's location.
# This means you can call the script from any working directory and it will
# still find meta_prompt.txt and the generated_prompts/ folder correctly.
SCRIPT_DIR = Path(__file__).parent.resolve()
META_PROMPT_PATH = SCRIPT_DIR / "meta_prompt.txt"
OUTPUT_DIR = SCRIPT_DIR / "generated_prompts"

# — Helpers


---



def slugify(text: str) -> str:
    """
    Convert a vague brief string into a safe filename.
    Example: "extract data from PDFs" -> "extract_data_from_pdfs"
    """
    text = text.lower().strip()
    text = re.sub(r"^[^w\s-]", "", text) # remove punctuation
    text = re.sub(r"[\s-]+", "_", text) # replace spaces/hyphens with
underscores
    text = re.sub(r"_+", "_", text) # collapse multiple underscores
    return text[:80] # truncate to avoid filesystem
limits

```

```

def load_meta_prompt() -> str:
    """
    Load meta_prompt.txt. Exits with a clear message if the file is
    missing.
    """
    if not META_PROMPT_PATH.exists():
        print(
            f"ERROR: meta_prompt.txt not found.\n"
            f"Expected it at: {META_PROMPT_PATH}\n"
            f"Create it by following Section 3 of guide.md.",
            file=sys.stderr,
        )
        sys.exit(1)

    content = META_PROMPT_PATH.read_text(encoding="utf-8").strip()

    if not content:
        print(
            f"ERROR: meta_prompt.txt is empty.\n"
            f"File path: {META_PROMPT_PATH}\n"
            f"Copy the content from Section 3 of guide.md into it.",
            file=sys.stderr,
        )
        sys.exit(1)

    return content


def get_api_key() -> str:
    """
    Retrieve the Groq API key from the environment. Exits if not set.
    """
    key = os.environ.get("GROQ_API_KEY", "").strip()
    if not key:
        print(
            "ERROR: GROQ_API_KEY is not set.\n"
            "Set it with:\n"
            "  export GROQ_API_KEY='gsk_your_key_here'    # Linux/Mac\n"
            "  set GROQ_API_KEY=gsk_your_key_here          # Windows CMD\n"
            "Or add it to a .env file in the meta/ directory.",
            file=sys.stderr,
        )
        sys.exit(1)
    return key


def save_output(slug: str, content: str) -> Path:
    """
    Save the generated prompt to generated_prompts/<slug>.txt.
    Creates the output directory if it doesn't exist.
    Returns the path where the file was saved.
    """
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

```

```

output_path = OUTPUT_DIR / f"{slug}.txt"

try:
    output_path.write_text(content, encoding="utf-8")
except PermissionError:
    print(
        f"ERROR: Cannot write to {output_path}\n"
        f"Check that you have write permissions to: {OUTPUT_DIR}",
        file=sys.stderr,
    )
    # Don't exit – the output was printed to stdout, so it's not lost
    return output_path

return output_path


def call_groq(client: Groq, system_prompt: str, user_brief: str) -> str:
    """
    Call the Groq API and return the generated text.
    Raises descriptive errors for each failure mode instead of swallowing
    them.
    """
    # The meta-prompt is the system message. The user brief is the user
    message.
    # This separation keeps the meta-prompt reusable across briefs without
    # re-sending it as part of user content (cleaner token accounting too).
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_brief},
    ]

    try:
        response = client.chat.completions.create(
            model=MODEL,
            messages=messages,
            max_tokens=MAX_TOKENS,
            temperature=TEMPERATURE,
        )
    except APIStatusError as e:
        # Covers 429 rate limit, 401 auth errors, 500 server errors, etc.
        print(
            f"ERROR: Groq API returned status {e.status_code}.\n"
            f"Message: {e.message}\n"
            f"Type: {type(e).__name__}",
            file=sys.stderr,
        )
        raise # re-raise so the caller can decide whether to retry or exit
    except APIConnectionError as e:
        print(
            f"ERROR: Could not connect to the Groq API.\n"
            f"Check your internet connection.\n"
            f"Details: {e}",
            file=sys.stderr,
        )

```

```

        raise
    except APITimeoutError as e:
        print(
            f"ERROR: Groq API request timed out.\n"
            f"The model may be overloaded. Try again in a few seconds.\n"
            f"Details: {e}",
            file=sys.stderr,
        )
        raise

    # Extract the text content from the response
    if not response.choices:
        print("WARNING: Groq returned an empty choices list.",
            file=sys.stderr)
        return ""

    message = response.choices[0].message
    content = message.content

    if content is None:
        print(
            "WARNING: Groq returned a message with null content.\n"
            "This can happen if the model triggered a content filter.",
            file=sys.stderr,
        )
        return ""

    return content.strip()

# — Main

def main():
    # 1. Validate CLI input
    if len(sys.argv) < 2:
        print(
            "Usage: python meta_prompter.py \"your vague brief here\"\n"
            'Example: python meta_prompter.py "extract data from PDFs"',
            file=sys.stderr,
        )
        sys.exit(1)

    # Join all positional args in case the user forgot to quote the brief.
    # "extract data from PDFs" passed without quotes becomes multiple argv
    items.
    user_brief = " ".join(sys.argv[1:]).strip()

    if not user_brief:
        print("ERROR: Brief cannot be empty.", file=sys.stderr)
        sys.exit(1)

    slug = slugify(user_brief)
    print(f"Brief    : {user_brief}")

```

```

print(f"Slug      : {slug}")
print(f"Model      : {MODEL}")
print("-" * 60)

# 2. Load prerequisites
api_key = get_api_key()
meta_prompt = load_meta_prompt()

# 3. Build the Groq client
client = Groq(api_key=api_key)

# 4. Call the API
print("Calling Groq API...")
try:
    generated = call_groq(client, meta_prompt, user_brief)
except (APIStatusError, APIConnectionError, APITimeoutError):
    # Error already printed in call_groq; exit cleanly
    sys.exit(1)

# 5. Handle empty/malformed response
if not generated:
    print(
        "WARNING: The API returned an empty or null response.\n"
        "Saving an empty file anyway. Check the Groq dashboard for
details.",
        file=sys.stderr,
    )
    generated = "[EMPTY RESPONSE – check API logs]"

# 6. Print to stdout
print("\n" + "=" * 60)
print(generated)
print("=" * 60 + "\n")

# 7. Save to file
output_path = save_output(slug, generated)
print(f"Saved to: {output_path}")

if __name__ == "__main__":
    main()

```

5. test_briefs.md

Copy this content into `meta/test_briefs.md`:

Test Briefs for Meta-Prompter

These are intentionally vague user requests – the kind a real stakeholder might

send in a Slack message. The meta-prompt must resolve the ambiguity and produce a prompt a developer could hand directly to an LLM with confidence.

Brief 1

****Brief:**** "extract data from PDFs"

****Why it's vague:**** It doesn't specify what **kind** of data (tables, text, metadata, form fields), what the PDFs contain (invoices, research papers, contracts), what format the extracted data should take, or what the downstream use case is.

Brief 2

****Brief:**** "classify support emails"

****Why it's vague:**** It omits the taxonomy of categories, how to handle ambiguous emails that could belong to multiple classes, what to do with spam, whether confidence scores are needed, and whether the output feeds a human or an automation.

Brief 3

****Brief:**** "write product descriptions"

****Why it's vague:**** It gives no information about the product category, target audience, tone (formal vs. playful), required length, SEO requirements, or which product attributes must always be included.

Brief 4

****Brief:**** "summarize legal documents"

****Why it's vague:**** Legal documents vary enormously (contracts, briefs, statutes, NDAs); it's unclear who the reader is (lawyer, client, executive), what the acceptable summary length is, and whether the model should flag risky clauses or just neutrally condense.

Brief 5

****Brief:**** "translate marketing copy"

****Why it's vague:**** It doesn't name the target language, doesn't specify whether to translate literally or localize culturally, doesn't say whether brand voice should be preserved or adapted, and doesn't address what to do with idioms, slogans, or trademarked terms.

6. Running the Tool

Run for all 5 briefs

Make sure you're inside the `meta/` directory before running these commands.

Linux/Mac (bash):

```
cd meta

briefs=(
  "extract data from PDFs"
  "classify support emails"
  "write product descriptions"
  "summarize legal documents"
  "translate marketing copy"
)

for brief in "${briefs[@}"; do
  echo "Running: $brief"
  python meta_prompter.py "$brief"
  echo ""
  sleep 2    # small pause to avoid Groq rate limits
done
```

Windows (PowerShell):

```
cd meta

$briefs = @(
  "extract data from PDFs",
  "classify support emails",
  "write product descriptions",
  "summarize legal documents",
  "translate marketing copy"
)
```

```
foreach ($brief in $briefs) {  
    Write-Host "Running: $brief"  
    python meta_prompter.py $brief  
    Write-Host ""  
    Start-Sleep -Seconds 2  
}
```

Windows (Command Prompt — one at a time):

```
python meta_prompter.py "extract data from PDFs"  
python meta_prompter.py "classify support emails"  
python meta_prompter.py "write product descriptions"  
python meta_prompter.py "summarize legal documents"  
python meta_prompter.py "translate marketing copy"
```

Verify output files were created

```
ls -lh generated_prompts/
```

Expected output:

```
-rw-r--r--  classify_support_emails.txt  
-rw-r--r--  extract_data_from_pdfs.txt  
-rw-r--r--  summarize_legal_documents.txt  
-rw-r--r--  translate_marketing_copy.txt  
-rw-r--r--  write_product_descriptions.txt
```

Preview a generated prompt in terminal

```
cat generated_prompts/extract_data_from_pdfs.txt
```

Or for paginated reading:

```
less generated_prompts/summarize_legal_documents.txt
```

Check that you can see all 6 section headers: **### ROLE**, **### CONTEXT**, **### TASK**, **### CONSTRAINTS**, **### FORMAT**, **### EXAMPLES**.

7. Gemini-as-Judge — Scoring Your Outputs

You'll use an LLM (Gemini or DeepSeek both work for this) to judge each generated prompt against a quality rubric. This simulates automated evaluation without needing human review for every file.

DeepSeek scoring prompt

For each of your 5 output files, paste the file's content into DeepSeek Chat along with this prompt:

DeepSeek Prompt:

You are an expert prompt quality evaluator. I'm going to give you a structured prompt generated by a meta-prompting system. Your job is to score it rigorously.

Evaluate the following prompt on these criteria:

1. **Sections present** — Does it contain all 6 required sections? The sections are: ROLE, CONTEXT, TASK, CONSTRAINTS, FORMAT, EXAMPLES. Answer yes/no for each.
2. **Quality score (1–5)** — Rate the overall quality of the prompt from 1 (completely inadequate) to 5 (production-ready). Use this scale:
 - 1: Missing multiple sections or fundamentally broken
 - 2: All sections present but most are weak/vague
 - 3: Usable but 2–3 sections need improvement
 - 4: Strong prompt with only minor gaps
 - 5: Excellent — a developer could use this immediately with confidence
3. **Missing or weak sections** — List any sections that are absent, too short, too vague, or fail to give the LLM enough guidance. Be specific: don't just say "EXAMPLES is weak" — say why.
4. **Notes** — One or two additional observations about what makes this prompt strong or what would most improve it.

Here is the prompt to evaluate:

[PASTE THE CONTENTS OF THE GENERATED PROMPT FILE HERE]

Respond in this exact format:

ROLE_PRESENT: yes/no
CONTEXT_PRESENT: yes/no
TASK_PRESENT: yes/no
CONSTRAINTS_PRESENT: yes/no
FORMAT_PRESENT: yes/no
EXAMPLES_PRESENT: yes/no
HAS_ALL_6: yes/no
QUALITY_SCORE: [1-5]
WEAK_OR_MISSING: [comma-separated list or "none"]
NOTES: [1-2 sentences]

Filling in judge_scorecard.csv

Open `meta/judge_scorecard.csv` and add the headers on the first line, then one row per brief.

CSV headers:

```
brief_id,brief_text,generated_prompt_file,has_all_6_sections,quality_score_1to5,missing_or_weak_sections,notes
```

Example filled-in row:

```
1,"extract data from
PDFs",generated_prompts/extract_data_from_pdfs.txt,yes,4,"EXAMPLES – only
one example provided; no edge case shown","ROLE and TASK sections are
strong; CONSTRAINTS could be more specific about handling scanned vs.
native PDFs"
```

Tips:

- Use `yes` or `no` (lowercase) for `has_all_6_sections`.
- Quote any field that contains commas.
- `brief_id` is 1–5, matching the brief numbering in `test_briefs.md`.
- Fill in a row for each of the 5 briefs after you've scored them.

Full CSV example (all 5 rows):

```
brief_id,brief_text,generated_prompt_file,has_all_6_sections,quality_score_1to5,missing_or_weak_sections,notes
1,"extract data from
PDFs",generated_prompts/extract_data_from_pdfs.txt,yes,4,"EXAMPLES – no
edge case","Strong TASK section; CONSTRAINTS lacks file format specifics"
2,"classify support
emails",generated_prompts/classify_support_emails.txt,yes,3,"FORMAT –
output schema unclear; CONSTRAINTS – only 3 rules","ROLE is excellent;
needs a clearer output structure"
3,"write product
descriptions",generated_prompts/write_product_descriptions.txt,yes,5,none,"
All sections strong; EXAMPLES section includes both a basic and a premium
product variant"
4,"summarize legal
documents",generated_prompts/summarize_legal_documents.txt,no,2,"EXAMPLES –
missing entirely; FORMAT – vague","Meta-prompt failed to generate EXAMPLES
section; ROLE is too generic"
5,"translate marketing
copy",generated_prompts/translate_marketing_copy.txt,yes,4,"CONTEXT –
doesn't mention source language","Good CONSTRAINTS section; EXAMPLES covers
a slogan edge case well"
```

8. findings.md

Copy this template into `meta/findings.md` and fill in the blanks after completing all 5 runs and scoring:

```
# Meta-Prompting Project – Findings
```

```
## What is meta-prompting?
```

```
<!-- Write 1–2 sentences in your own words. What is meta-prompting and why does it exist? -->
```

```
_Your answer here._
```

```
---
```

```
## Meta-prompting vs. manual prompting
```

```
| Situation | Approach |
|-----|-----|
| Meta-prompting is better when... | The task type repeats across many different user briefs (same structure needed each time) |
| Meta-prompting is better when... | The user brief is consistently vague and the structured output template is fixed |
| Manual prompting is faster when... | You have a one-off task with a very specific, already-clear requirement |
| Manual prompting is faster when... | The task is simple enough that a 2-sentence prompt works reliably |
| _(add your own)_ | |
| _(add your own)_ | |
```

```
---
```

```
## Which of my 5 generated prompts was strongest, and why?
```

```
<!-- Name the brief, and explain what made the generated prompt effective. Reference specific sections (e.g., "The CONSTRAINTS section was unusually detailed because...") -->
```

```
_Your answer here._
```

```
---
```

```
## Which was weakest? What did the meta-prompt fail to resolve?
```

```
<!-- Name the brief. What ambiguity remained in the output? What rule could you add to meta_prompt.txt to fix this? -->
```

```
_Your answer here._
```

```
---
```

One thing I would change about my meta-prompt after testing

<!-- This should be a concrete, actionable change – not "make it better".
Example: "I would add a rule to the EXAMPLES section requiring at least one failure case / edge case." -->

Your answer here.

9. learnings.md

Copy this template into `meta/learnings.md`. Fill in one entry for every bug, unexpected behavior, or confusion you hit during the project. Add as many entries as needed.

Meta-Prompting Project – Debugging Log

Entry 1

****What broke:****

<!-- Paste the exact error message or describe the unexpected output -->

****Which file / line caused it:****

<!-- e.g., "meta_prompter.py, line 47 – load_meta_prompt()" -->

****What I tried:****

<!-- List the things you did to diagnose or fix it -->

****What fixed it (or what I still don't understand):****

<!-- Be honest – "I still don't know why this happens" is a valid entry -->

****One principle I'll remember from this:****

<!-- One sentence. Make it general enough to apply to future projects -->

Entry 2

****What broke:****

****Which file / line caused it:****

****What I tried:****

****What fixed it (or what I still don't understand):****

****One principle I'll remember from this:****

```
<!-- Add more entries as needed -->
```

10. Troubleshooting

`meta_prompt.txt` not loading (wrong working directory)

Symptom:

```
ERROR: meta_prompt.txt not found.  
Expected it at: /some/wrong/path/meta_prompt.txt
```

Cause: You ran the script from a directory other than `meta/`, and `Path(__file__).parent` resolved to a different location than expected — or the file simply doesn't exist yet.

Fix:

```
# Confirm your current directory  
pwd  
  
# Make sure you're in meta/ or that meta_prompt.txt exists at the path  
# shown in the error  
ls meta/meta_prompt.txt  
  
# Run from the meta/ directory or use an absolute path  
cd meta && python meta_prompter.py "your brief"
```

The script uses `Path(__file__).parent` so it always looks relative to `meta_prompter.py`, not your shell's current directory. If the error path is wrong, it means `meta_prompter.py` itself is in the wrong location.

Groq returns a 429 rate limit error

Symptom:

```
ERROR: Groq API returned status 429.  
Message: Rate limit exceeded.
```

Cause: Free Groq tier has per-minute token and request limits. Running all 5 briefs back-to-back without pausing triggers this.

Fix:

Add `sleep 3` (or more) between calls in your loop:

```
for brief in "${briefs[@]"; do
  python meta_prompter.py "$brief"
  sleep 5    # increase to 10 if 429s persist
done
```

You can also check your current usage at console.groq.com.

Generated prompt is missing one or more of the 6 sections

Symptom: The output file exists but when you `cat` it, one or more of `### ROLE`, `### CONTEXT`, `### TASK`, `### CONSTRAINTS`, `### FORMAT`, `### EXAMPLES` is absent.

Cause: The model didn't follow the meta-prompt instructions. This happens when:

- `TEMPERATURE` is set too high (above 0.7), making the model more creative and less rule-following.
- The meta-prompt instructions for that section are too vague.
- The model was cut off early due to `max_tokens` being too low.

Fix options:

1. Lower temperature: change `TEMPERATURE = 0.4` to `TEMPERATURE = 0.2` in `meta_prompter.py`.
 2. Increase `MAX_TOKENS` from `2048` to `3000` if you suspect truncation.
 3. Strengthen the relevant section rules in `meta_prompt.txt`. Add: *"You must include this section even if it seems obvious. Omitting any section is a critical failure."*
 4. Re-run the failing brief after making changes.
-

Output file not being saved (permissions or path issue)

Symptom:

```
ERROR: Cannot write to /meta/generated_prompts/extract_data_from_pdfs.txt
```

Cause: The `generated_prompts/` directory doesn't exist, or your user doesn't have write permission to it.

Fix:

```
# Create the directory manually
mkdir -p meta/generated_prompts

# Check permissions
ls -la meta/
```

```
# If permissions are wrong
chmod 755 meta/generated_prompts
```

Note: the script will still print the generated prompt to stdout even if the file save fails — so your output is not lost. Copy it from the terminal if needed.

CLI argument with spaces causing shell parsing issues

Symptom: Running:

```
python meta_prompter.py extract data from PDFs
```

Generates a slug like `extract` because only `sys.argv[1]` was received, and `data`, `from`, `PDFs` were treated as separate arguments.

Cause: Shell word-splitting. The brief must be quoted.

Fix: Always quote the brief:

```
python meta_prompter.py "extract data from PDFs"
```

The script also joins all argv items as a fallback (`" ".join(sys.argv[1:])`), so unquoted multi-word briefs will still work in most cases — but you should quote them anyway for reliability and to avoid issues with special characters like `&`, `|`, `>`, or `*`.

End of guide. If something isn't covered here, add it to your `learnings.md` and bring it to your next mentorship check-in.